

Multi-Agent Reinforcement Learning Aided Computation Offloading in Aerial Computing for the Internet-of-Things

Zeyu Qin, *Student Member, IEEE*, Haipeng Yao[✉], *Senior Member, IEEE*, Tianle Mai[✉], *Member, IEEE*, Di Wu, *Student Member, IEEE*, Ni Zhang, and Song Guo[✉], *Fellow, IEEE*

Abstract—LEO satellite networks have become a necessary supplement to terrestrial networks aiming to provide worldwide, ubiquitous connectivity, especially in complicated areas (e.g., mountains, oceans, and disaster areas) where terrestrial network infrastructures are typically sparingly distributed or unavailable. However, the increasing computation-intensive Internet-of-Things (IoT) applications (e.g., real-time remote monitoring, intelligent transportation) require not only efficient and reliable communication but also massive computing capabilities. Constrained by the battery and computing resources, the computing tasks and data of applications have to be transmitted to remote cloud servers. This bandwidth limitation and high transmission delay in LEO networks will reduce the quality-of-service (QoS) of IoT applications. Recently, the combination of LEO networks and edge computing (i.e., Satellite Mobile Edge Computing, SMEC) offers significant opportunities to address these problems. The IoT devices can directly get the computing resources directly from satellites rather than remote servers, thus avoiding long-distance transmission. Considering the resource constraints on satellites, offloading policy plays a crucial role in whole system performance. In this paper, we design a hybrid offloading architecture, which applies a centralized training and distributed execution framework. Also, we propose a multi-agent actor-critic reinforcement learning algorithm, where a centralized “critic” is augmented with the global network state to ease the training procedure of distributed user equipments (UE) by evaluating the benefits of their decisions, while the UEs can adjust their policies according to the critic’s evaluation and choose their own decisions relying on their observations.

Index Terms—Aerial computing, computation offloading, deep reinforcement learning, mobile edge computing, multi-agent system

1 INTRODUCTION

THE past few years have witnessed an exponential growth of various satellites as well as compelling space applications ranging from intelligent transport, and disaster rescue to military surveillance [1]. Satellite networks, particularly low earth orbit (LEO) satellite networks, have become an important complement to terrestrial networks due to their unique benefits, such as global coverage, high resilience, and excellent bandwidth availability. The satellite network could meet global demands for “ubiquitous connection” of users,

particularly in rough terrains like hills and seas [2]. It can also help with emergency communication if the terrestrial network infrastructure is destroyed by natural disasters. Many countries have recently launched over 8800 satellites into orbit, and the number continues to rise [3]. For example, SpaceX Starlink plans to launch 42000 satellites to supply people in need with communication assistance all over the world [4]. Hence, we can infer that LEO satellite networks have incomparable superiority beyond terrestrial networks, realize seamless coverage globally, and display an essential part of daily human life.

However, the growing number of computation-intensive applications (e.g., real-time remote monitoring, intelligent transportation) requires not only efficient and reliable communication but also vast computational power. While the processing capabilities of IoT devices have substantially improved, they are still constrained by the battery and computing resources, especially the lightweight IoT devices [5]. The computation tasks in remote areas must be transmitted to the remote cloud servers for further processing and storage through LEO satellite networks. Considering the large transmission delay and limited bandwidth in the LEO networks, this processing architecture faces challenges in terms of performance, scalability, and efficiency, making it complicated to satisfy the real-time demands of some latency-sensitive applications (e.g., self-driving, hotspot tracking) [6].

- Zeyu Qin, Haipeng Yao, Tianle Mai, and Di Wu are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China. E-mail: {qinzeyu724666925, machealmal@gmail.com, yaohaipeng@bupt.edu.cn, wudi828282@hotmail.com}.
- Ni Zhang is with the Sixth Research Institute of China Electronic Corporation, Beijing 100846, China. E-mail: zhangni@ncse.com.cn.
- Song Guo is with the Department of Computing, Hong Kong Polytechnic University, Hung Hom, Hong Kong. E-mail: song.guo@polyu.edu.hk.

Manuscript received 23 January 2022; revised 2 June 2022; accepted 8 July 2022. Date of publication 13 July 2022; date of current version 12 June 2023.

This work was supported in part by the funding from the National Key R&D Program of China under Grant 2018YFB1800805, in part by Artificial Intelligence and Smart City Joint Laboratory under Grants BUPT-TGSTII and B2020001, and in part by Future Intelligent Networking and Intelligent Transportation Joint Laboratory under Grants BUPTCTIC and B2019007.

(Corresponding author: Haipeng Yao.)

Digital Object Identifier no. 10.1109/TSC.2022.3190562

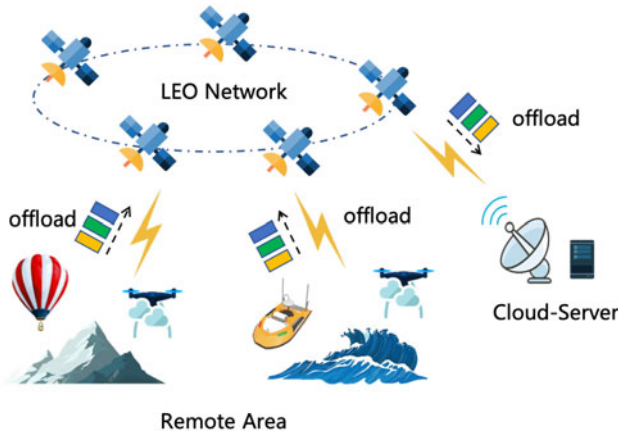


Fig. 1. Computation offloading architecture in SMEC.

The combination of LEO networks and edge computing (SMEC) networks has recently received a large quantity of attention from both academia and industry [7]. As shown in Fig. 1, the SMEC architecture could take full advantage of the computing resources of satellites, where UEs can directly get the computing resources directly from satellites rather than remote servers, thus avoiding long-distance transmission. While this offloading mechanism can enhance application performance and reduce power consumption for IoT devices, it is not a panacea for improving performance [8]. On the one hand, satellites' computing and storage capacities are still limited, so that it is impractical to offload all tasks to satellites. On the other hand, the offloading process also incurs extra power consumption (i.e., transmitting power) and transmission delays (e.g., fronthaul link transmission delay). Therefore, how to design effective computation offloading schemes plays a crucial role.

Current offloading schemes largely rely on the centralized framework, where a centralized scheduler constantly gathers global system information and then calculates the optimal offloading decisions [9]. While the advantages of centralized architecture are obvious (e.g., quick convergence, global optimum), it incurs too much communication and computation overhead, especially to the large scale LEO network [10]. Whenever the requirements of UEs changes, the scheduler has to collect massive information across the entire system and recalculate the offloading decision. Besides, when the scheduler node experience a hardware failure or is attacked, the entire system may be paralyzed [11].

To handle these challenges, there has been an increasing amount of literature focusing on distributed offloading schemes. Compared to the centralized schemes, each UE can make the offloading decision only according to its local observation [12]. However, as a distributed system, learning from nodes with only local observation and control is difficult, especially when the goal is global optimization [13]. The distributed solutions are confronted with the seriously non-convergence problem.

Therefore, in this paper, we design a hybrid computation offloading architecture and adopt a centralized training and distributed execution framework [14]. In our architecture, we introduced a centralized platform to gather global information and all UEs' actions to facilitate the learning procedure whilst all distributed actors are able to perform

functions independently, relying on their local observations. Benefiting from the framework, multiple UEs can work cooperatively, while each UE can still select and perform its action only according to their local observation independently. In order to compare our algorithm with other state-of-the-art solutions, we performed extensive and in-depth simulations to evaluate our proposed algorithm.

The main contributions of our paper are summarized as follows.

- We consider a three-tier computation offloading architecture in LEO satellite networks. The offloading optimization problem is formulated with two objectives, minimizing task delay and reducing energy consumption.
- We proposed a hybrid computation offloading architecture and adopted a centralized training and distributed execution framework. The centralized platform takes advantage of global system states to ease the training procedure of the distributed UEs whilst the distributed UEs can make decisions according to their local observations.
- We conducted extensive simulations to illustrate the convergence and efficiency of our algorithm.

The rest of this paper is organized as follows. We discuss the related work in Section 2. Then, we discuss the system model and problem formulation in Section 3. In Section 4, we proposed the multi-agent deep reinforcement learning algorithm aided computation offloading problem. In Section 5, the simulation result is presented to demonstrate the feasibility and performance of our algorithm. and the conclusion is in Section 6.

2 RELATED WORK

Lately, a significant number of literature has been published on computation offloading in satellite networks. In the following, we would like to talk about the related works from the aspect of satellite networks and machine learning-based schemes.

2.1 Computation Offloading in Satellite Networks

Recently, a significant and growing body of literature has looked into computation offloading in satellite networks. And lots of work has been done on the ground-air-space merged network. In [15], Alsharoa *et al.* proposed a frequency partitioning algorithm to optimize resource allocations in the short-term stage and a shrink-and-realign algorithm to optimize locations of high-altitude platforms in the long-term stage. In [16], Cheng *et al.* proposed a corporate virtual machine (VM) assignment mechanism in the ground-air-space merged network to allocate the computational resources to the virtual machines efficiently. In [17], Liu *et al.* introduced a task-oriented approach to the ground-air-space unified network to decrease response time and implement smart networking capacities. Meanwhile, a large number of scholars have achieved important progress in the comprehensive optimization of various conditions of the satellite network. In [18], Han *et al.* took the dynamic and complex characteristics of space-based information networks into consideration and introduced a scheduling algorithm

TABLE 1
List of Main Notations

Parameter	Definition
M	Number of LEOs
K	Number of UEs
$p_{i,n}^S$	Transmission power when offloading $T_{i,n}$ to LEOs
$p_{i,n}^M$	Transmission power when offloading $T_{i,n}$ to cloud server
$R_{i,j}^S$	Transmission rate between UE_i and LEO_j
R_i^M	Transmission rate between UE_i and cloud server
D_S^{tr}	Transmission delay between UE_i and LEO_j
D_M^{tr}	Transmission delay between UE_i and cloud server
E_S^{tr}	Transmission energy consumption between UE_i and LEO_j
E_M^{tr}	Transmission energy consumption between UE_i and cloud server
D_l^{com}	Computation delay for processing the task locally
D_S^{com}	Computation delay for processing the task on LEO_j
D_M^{com}	Computation delay for processing the task on cloud server
E_l^{com}	Computation energy consumption for processing the task locally
E_S^{com}	Computation energy consumption for processing the task on LEO_j
E_M^{com}	Computation energy consumption for processing the task on cloud server
$C_{i,n}^l$	Total cost for processing $T_{i,n}$ locally
$C_{i,j,n}^S$	Total cost for offloading $T_{i,n}$ to LEO_j
$C_{i,n}^M$	Total cost for offloading $T_{i,n}$ to the cloud server

for LEOs. In [19], Zhang *et al.* designed an effective network virtualization technique to jointly allocate the network resources to promote the communication quality of the mobile equipment. In [20], Wang *et al.* introduced a computation offloading framework and designed an iterative approach to seek the Nash equilibrium. In [21], Guo *et al.* proposed an intelligent computation offloading scheme to reduce the system's energy cost by jointly optimizing spectrum, and energy, including computation resource allocation. In [22], Guo *et al.* proposed an offline dynamic programming approach to achieve optimal energy efficiency, and the approach shows good adaptability. Current solutions mainly focus on the task offloading from satellites to ground devices and do not consider the need for offloading computation tasks from IoT devices in remote areas.

2.2 Computation Offloading With Machine Learning

Besides, we notice that lots of works have studied multi-tier task offloading problems. Current solutions can be roughly divided into two categories: centralized and distributed. The centralized algorithm decides the offloading location based on global information and it is thus easy to converge to the global optimum. In [23], Huang *et al.* proposed applying deep reinforcement learning to solve the binary computation offloading problem in radio-powered mobile edge computing (MEC). In [24], Song *et al.* decomposed the computation offloading problem in satellite networks into two-layered subproblems and proposed the resource allocation algorithm. The outcomes of the proposed simulation revealed that the introduced algorithm betters complete offloading and local computing entirely. In [25], Chen *et al.* considered the multihop computation offloading problem

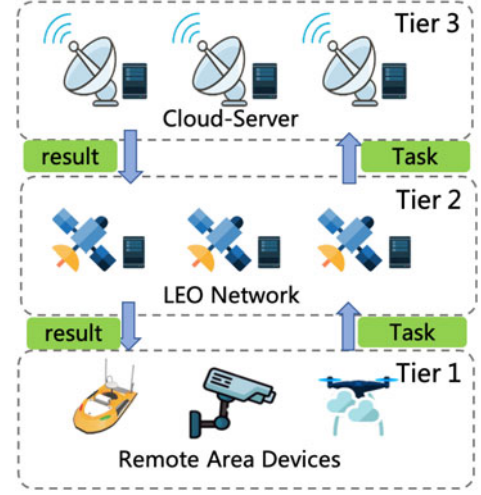


Fig. 2. Offloading system model.

and proposed a high-efficiency distributed algorithm to offload computation-intensive tasks in blockchain-empowered IoT to the MEC servers based on potential game theory and prove to reach a Nash equilibrium. In [26], Zhao *et al.* constructed a three-tier task offloading algorithm to optimize the task computing delay and energy cost based on a discrete particle swarm optimization algorithm. However, the centralized controller needs to collect a lot of information, which will lead to excessive communication overhead. This disadvantage is more obvious in large-scale LEO networks. Therefore, some research focuses on the distributed architecture, where the agents can decide the offloading location independently and immediately. In [27], Zhan *et al.* proposed applying proximal policy optimization to help making decisions on how to offload tasks and when to offload tasks. In [28], Liu *et al.* proposed a distributed task transfer method based on the counterfactual multi-agent reinforcement learning algorithm to solve the task transfer problem caused by mobility of distributed users. However, in distributed algorithms, agents often face the problem of slow convergence and difficulty in achieving global optimality.

3 SYSTEM MODEL AND PROBLEM FORMULATION

In the section, we firstly introduce the SMEC architecture. Then, we discuss the system model and problem formulation. For better perception, we list the notations that appeared in the paper in Table 1.

3.1 Network Model

In this paper, we consider a SMEC system consisting of multiple LEO satellites and cloud servers [29]. As shown in Fig. 2, a three-tier SMEC is presented, where *tier1* consists of several $UEs = \{UE_1, UE_2, \dots, UE_K\}$, *tier2* consists of multiple $LEOs = \{LEO_1, LEO_2, \dots, LEO_M\}$, and *tier3* consists of several cloud servers. We make the assumption that every UE can directly communicate with the LEO satellite, and each LEO satellite can directly connect to the cloud server. The MEC servers are deployed at each satellite, which has varying computing and storage capabilities. Notably, we consider the optimization of computation task offloading decisions on small time scales in this paper. Therefore, we

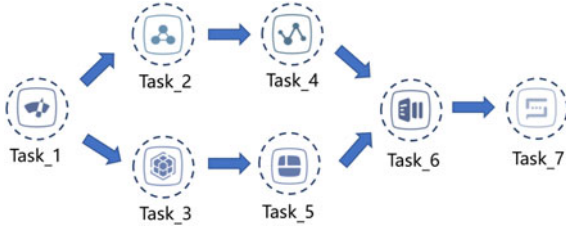


Fig. 3. Dependency among computational tasks.

treat the satellite as quasi-static and ignore the task handover between the satellites.

3.2 Task Dependency Model

We assume that the applications on UEs are constantly generating computing tasks. The task of UE_i is denoted as T_i , which can be split into a sequence of sub-tasks (e.g., user interaction task, image compression task, and sensor data processing task), and each sub-task is interdependent on the others. The dependencies indicate what needs to be done before a sub-task can begin, which can be divided into two categories: timing dependency and data dependency. The timing dependency refers to the relationship between nearby computing tasks in terms of time sequence. One sub-task cannot be completed until the sub-tasks on which it is dependent have been completed. Data dependency refers to the relationship between nearby computing sub-tasks in terms of data transport. The computation result of the first sub-task is required for the processing of the second.

In this section, the dependencies are denoted by a directed acyclic graph (DAG) $g = (T_i, L)$, where $T_i = \{T_{i,1}, T_{i,2}, T_{i,3}, \dots, T_{i,n}\}$ represents a set of sub-tasks and $L = \{l(T_{i,p}, T_{i,q}) | T_{i,p}, T_{i,q} \in T\}$ represents the set of dependencies among the computing sub-tasks. $l(T_{i,p}, T_{i,q}) = 0$ denotes there is no dependency relationship between $T_{i,p}$ and $T_{i,q}$ while $l(T_{i,p}, T_{i,q}) = 1$ denotes that $T_{i,q}$ could not be processed before $T_{i,p}$ is executed. As shown in Fig. 3, the task could be split into seven sub-tasks, and the sub-task t_1 must first be processed, and the sub-task t_7 must be processed at last. For every step, the tasks that can be processed form an available task set M and when M is not empty, the UEs will choose one of them to process.

Besides, the n -th sub-task of UE_i is denoted as $T_{i,n} = \{r_{i,n}, c_{i,n}, t_{i,n}^{max}\}$, where $r_{i,n}$ is the data size, $c_{i,n}$ is the total computation resource requirement, which is quantified as the number of CPU cycles in this paper, and $t_{i,n}^{max}$ is the maximum latency constraints.

Besides, we denote the offloading decision of UE_i for $T_{i,n}$ as $X_{i,n} = \{x_{i,n,0}, x_{i,n,1}, x_{i,n,2}\}$, where $x_{i,n,0} = 1$ means to execute the sub-task locally, $x_{i,n,1} = 1$ means to offload the computing sub-task to the satellites, and $x_{i,n,2} = 1$ means to offload the sub-task to the cloud server. It is worth noting that every UE can only offload the sub-task to a certain satellite if the UE decided to offload the task to satellites. When $X_{i,n,1} = 1$, the UE will offload the sub-task to the nearest satellites. Correspondingly, we define a transmission power vector $P_{i,n} = \{p_{i,n}^S, p_{i,n}^M\}$, where $p_{i,n}^S$ denotes the transmission power when the sub-task is offloaded to the satellites, and $p_{i,n}^M$ denotes when the sub-task is offloaded to the cloud server.

3.3 Communication Model

In this paper, we consider two offload schemes as discussed above. The UEs can directly offload the task to the satellites or to the cloud server [30]. To calculate the end-to-end latency and energy consumption, we present the communication model in this part. Since the computation result's data size is much smaller than that of the task, we do not consider the result's transmission delay and the transmission power [31]. We denote the transmission rate between the i -th user equipment UE_i and m -th LEO LEO_m as $R_{i,m}^T$, which can be denoted as:

$$R_{i,m}^T = W_{i,m} \log_2 \left(1 + \frac{p_{i,s}^S g_{i,s}}{\sigma^2 + \sum_{l=1}^M \sum_{k=1, k \neq i}^K p_{k,l}^S g_{k,l}} \right), \quad (1)$$

where $W_{i,m}$ is the link bandwidth of UE_i to LEO_m , σ^2 is the noise power, and $g_{i,m}$ is the channel gain. Then, the transmission delay can be calculated by:

$$D_T^{tr} = \frac{r_{i,n}}{R_{i,j}^S}, \quad (2)$$

where $r_{i,n}$ is the data size of the task. Besides, the transmission energy consumption can be formulated as:

$$E_T^{tr} = p_{i,s}^S \times D_S^{tr}. \quad (3)$$

In contrast, when UE_i decides to offload $T_{i,n}$ to *cloudserver*, the transmission rate of UE_i to LEO_j can be formulated as:

$$R_{i,j}^S = W_{i,j} \log_2 \left(1 + \frac{p_{i,s}^M g_{i,s}}{\sigma^2 + \sum_{l=1}^M \sum_{k=1, k \neq i}^K p_{k,l}^S g_{k,l}} \right), \quad (4)$$

where $W_{i,j}$ denotes the link bandwidth of UE_i to LEO_j and $g_{i,s}$ is the channel gain. Similarly, we can calculate the transmission rate of LEO_j to *cloudserver* as:

$$R_j^M = W \log_2 \left(1 + \frac{p_{s,m} g_{s,m}}{\sigma^2 + \sum_{q=1}^N \sum_{l=1, l \neq s}^M p_{l,q} g_{l,q}} \right). \quad (5)$$

In full-duplex two-hop computation offloading, the LEO serves as the relay node and is capable of transmitting and receiving at the same time. In this way, the transmission rate of the UE is limited by the lower transmission rate of the two links. Therefore, the transmission rate R_i^M of UE_i to *cloudserver* can be re-formulated as:

$$R_i^M = \text{Min}(R_{i,j}^S, R_j^M). \quad (6)$$

Then, the transmission delay between UE_i to *cloudserver* can be calculated by:

$$D_M^{tr} = \frac{r_{i,n}}{R_i^M}. \quad (7)$$

In addition, the transmission power consumption can be formulated by:

$$E_M^{tr} = p_{i,s}^M \times D_M^{tr}. \quad (8)$$

3.4 Computation Model

We next discuss the computing task, processing model. When a task is executed locally, the processing delay of the task could be calculated by:

$$D_l^{com} = \frac{c_{i,n}}{f_i^l}, \quad (9)$$

where $c_{i,n}$ denotes the total computation resource requirement, f_i^l denotes local computing resources of UE_i . The energy consumption can be formulated as:

$$E_l^{com} = e_i^l \times c_{i,n}, \quad (10)$$

where e_i^l is the energy consumption of the unit computing resource of UE_i .

When UE_i decides to offload the task to LEO_j , the processing delay can be calculated by:

$$D_S^{com} = \frac{c_{i,n}}{f_{i,j}^S}, \quad (11)$$

where $f_{i,j}^S$ is the computing resources allocated to UE_i . The computation energy consumption can be calculated by:

$$E_{i,j,n}^{com} = e_j^S \times c_{i,n}, \quad (12)$$

where e_j^S represents the energy consumption per unit computing resource of LEO_j .

When UE_i decides to offload the task to *cloudserver*, the processing delay can be calculated by:

$$D_M^{com} = \frac{c_{i,n}}{f_i^M}, \quad (13)$$

where f_i^M represents the computing resources allocated to UE_i . The computation energy consumption can be calculated by:

$$E_{i,n}^{com} = e_j^M \times c_{i,n}, \quad (14)$$

where e_j^M represents the energy consumption per unit computing resource of *cloudserver*.

3.5 Problem Formulation

In this paper, the offloading optimization problem can be formulated with two objectives, minimizing task delay and reducing energy consumption. And to evaluate the offloading strategy, the cost function is defined as follows. When UE_i decides to execute $Task_n$ locally, the cost can be defined as: $C_{i,n}^l = \phi_1 D_l^{com} + \phi_2 E_l^{com}$; when UE_i decides to offload $Task_n$ to LEO_j , the cost can be calculated by: $C_{i,j,n}^S = \phi_3 (D_S^{tr} + D_S^{com} + D_T^{tr}) + \phi_4 (E_S^{tr} + E_S^{com} + E_T^{tr})$; when UE_i decides to offload $Task_n$ to *cloudserver*, the cost can be defined as: $C_{i,n}^M = \phi_5 (D_M^{tr} + D_M^{com} + D_T^{tr}) + \phi_6 (E_M^{tr} + E_M^{com} + E_T^{tr})$, where $\phi_1, \phi_2, \phi_3, \phi_4, \phi_5, \phi_6$ are the weights for computation and transmission. Then we can define the cost function of UE_i as:

$$Cost_i = x_{i,n,0} C_{i,n}^l + x_{i,n,1} C_{i,j,n}^S + x_{i,n,2} C_{i,n}^M, \quad (15)$$

Therefore, the offloading optimization problem of task offloading can be formulated as:

$$\min \sum_{i=1}^{Task_{num}} \sum_{n=1}^N x_{i,n,0} C_{i,n}^l + x_{i,n,1} C_{i,j,n}^S + x_{i,n,2} C_{i,n}^M, \quad (16)$$

st.

$$\begin{aligned} C_1 : & x_{i,n,0}, x_{i,n,1}, x_{i,n,2} \in \{0, 1\} \\ C_2 : & x_{i,n,0} + x_{i,n,1} + x_{i,n,2} = 1 \\ C_3 : & x_{i,n,0} D_l^{com} + x_{i,n,1} (D_S^{tr} + D_S^{com}) + \\ & x_{i,n,2} (D_M^{tr} + D_M^{com}) \leq t_{i,n}^{max} \\ C_4 : & p_{i,n}^S, p_{i,n}^M \leq P_i^{max} \\ C_5 : & \sum_{i=1}^{K_j} x_{i,n,1} f_{i,j}^S < f_{S,j}^{max} \\ C_6 : & \sum_{i=1}^K x_{i,n,2} f_i^M < f_M^{max} \end{aligned} \quad (17)$$

where C_1 indicates that the offloading decisions variables are 0-1 binary, C_2 indicates that each user can choose only one offloading decision for each task, C_3 indicates that the delay of each offloading method cannot exceed the maximum acceptable delay of the task, C_4 indicates that the UE's transmission power couldn't go beyond the maximum power for transmission, C_5 indicates that the computational resource occupied cannot exceed the total computational resources of the LEO when multiple UEs offload tasks to the same LEO, and C_6 indicates that the computation resource occupied couldn't go beyond the total computing resources of the cloud server when multiple UEs offload tasks to cloud servers.

4 MULTI-AGENT DEEP REINFORCEMENT LEARNING ALGORITHM

In this section, we first formulate the cooperation of UEs as a Partially Observable Markov Decision Process (POMDP). Then, we design a multi-agent reinforcement learning-based algorithm to solve the computation offloading problem.

4.1 Partially Observable Markov Decision Process

We can model the multiple computation tasks offloading decision process on the UEs as a partially observable Markov decision process (POMDP) [32]. Formally, a POMDP model can be described as a 6-tuple

$$P = (S, A, T, R, O, \gamma),$$

where $S = \{s_1, s_2, \dots, s_n\}$ denotes the state space, $A = \{a_1, a_2, \dots, a_m\}$ denotes the action space, T denotes the conditional transition probabilities $T(s'|s, a)$, R denotes the immediate reward function $R: S \times A \rightarrow R$, $O = \{o_1, o_2, \dots, o_n\}$ denotes the agent's local observation, and $\gamma \in [0, 1]$ is the discount factor. For every step, every UE obtains its own observation o_i and then chooses its action a_i based on current policy π_i . After each UE performing its actions, the environment would produce an immediate reward R . Meanwhile, the state of s will change to a new state s' . The object of RL agents is to learn a strategy that will maximize the discounted cumulative reward $R_i = \sum_{t=0}^T \gamma^t r_i^t$.

Specifically, in the proposed scenario, the observation space could be defined as $o_{i,n} = \{P_{i,n}^{max}, e_i^l, f_i^l, g_{i,j}, T_{i,n}\}$, where

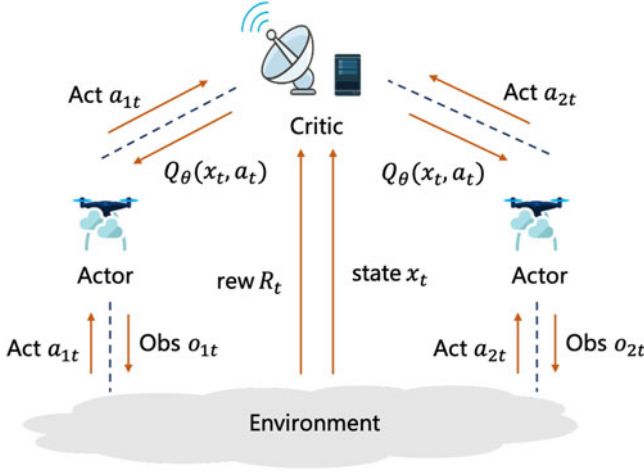


Fig. 4. Multi-agent system of computation offloading.

P_i^{max} represents the maximum transmission power of UE_i , e_i^l represents the energy consumption of per unit computing resource of UE_i , f_i^l represents the total amount of computing resources of UE_i , $g_{i,j}$ represents the channel gain of the wireless link between UE_i and LEO_j , and $T_{i,n} = \{r_{i,n}, c_{i,n}, t_{i,n}^{max}\}$ represents the requirements of the task. The action space of $actor_i$ can be defined as $action_{i,n} = \{x_{i,n}, p_{i,n}\}$, where $x_{i,n} = \{x_{i,n,0}, x_{i,n,1}, x_{i,n,2}\}$ represents the user's offloading decision, $p_{i,n} = \{p_{i,n}^s, p_{i,n}^m\}$ indicates the transmission power, $x_{i,n,0}, x_{i,n,1}, x_{i,n,2} \in \{0, 1\}$ indicates the three locations can use to execute the computation tasks, and $\{p_{i,n}^s, p_{i,n}^m\} \in (0, p_i^{max})$ denotes the transmission power when offloading tasks to satellites and cloud servers. It's worth mentioning that the three actions in the action space is not optional at all times. For example, when the computational resource is not sufficient to complete the task, the $x_{i,n,2}$ could not be set to 1. In addition, we set the reward as $r_{i,n} = -(x_{i,n,0}c_{i,n}^l + x_{i,n,1}c_{i,n}^s + x_{i,n,2}c_{i,n}^m)$. The goal of each agent is learning a strategy to choose the best action $\{x_{i,n}, p_{i,n}\}$ according to the current state to maximize the discounted reward $R_i = \sum_{t=0}^{\infty} \gamma^t r_i^t$.

4.2 Multi Agent Deep Reinforcement Learning Algorithm

Learning in the multi-agent system is far more complicated than in the single-agent system, caused by the non-stationary problem [33]. In this paper, we design a multi-agent deep reinforcement learning algorithm (MADRL) to enable each UE to get the optimal offloading strategy. MADRL adopts the actor-critic-based framework. A critic is used to evaluate the action chosen by the actor function, where the critic is responsible for estimating the value function, and the actor is responsible for updating the policy distribution according to the critic's advise. As shown in Fig. 4, the MADRL architecture consists of multiple distributed actors and a logically centralized critic. In this paper, the actors are deployed at each UE, and the centralized critic platform is deployed at the cloud servers.

During the execution, each actor observes the environment and then selects the corresponding action according to the current strategy. As mentioned above, it is unrealistic to enable each UE to obtain global network information in the actual deployment. Therefore, the observation of each UE only contains local information. Besides, each UE does not

know other UEs' actions and the MEC servers' current load. This is because of the assumption that each UE may not be willing to share its actions with other UEs. Therefore, $Actor_i$'s local observation $o_{i,n}$ contains the computational capability and energy consumption per unit computing resource of the UE, the required resource of the task, and the current conditions of the wireless channel.

We consider N agents with policies $\mu = [\mu_1, \mu_2, \dots, \mu_N]$, which parameterized by $\theta = [\theta_1, \theta_2, \dots, \theta_N]$. For each actor i , the gradient of the expected reward for a certain agent can be calculated by:

$$\nabla_{\theta_i} J(\mu_i) = E_{x,a \sim D} [\nabla_{\theta_i} \mu_i(a_i | o_i) \text{Grad} Q],$$

$$\text{where } \text{Grad} Q = \nabla_{a_i} Q_i^{\mu}(x, a_1, \dots, a_n) |_{a_i = \mu_i(o_i)}, \quad (18)$$

where the $x = (o_1, o_2, \dots, o_n)$ is the global network information and the $a_1, \dots, a_n$ is the joint action. Note that a replay buffer $D = \{x, x', a_1, \dots, a_n, r_1, \dots, r_n\}$ is adopted to handle the non-stationary data distribution issue.

In Eq. (18), the $Q_i^{\mu}(x, a_1, \dots, a_n)$ is the centralized "critic" Q-function, which takes as input the globe network information $x = (o_1, o_2, \dots, o_n)$ and joint actions $a_1, \dots, a_n$. In our architecture, the centralized critic is deployed at the cloud server, which is used to guide each actor to work in a cooperative way. The loss function of $Q_i^{\mu}(x, a_1, \dots, a_n)$ can be calculated by:

$$L(\theta_i) = E_{x,a,r,x'} [(Q_i^{\mu}(x, a_1, a_2, \dots, a_n) - y)^2],$$

$$\text{where } y = r_i + \gamma Q_i^{\mu'}(x', a'_1, a'_2, \dots, a'_N). \quad (19)$$

As shown in Eq. (19), we adopt the target network $Q_i^{\mu'}$ to make the learning processing more stable, where $\mu' = \{\mu'_1, \mu'_2, \dots, \mu'_N\}$ denotes the parameter of delayed updated target network. Besides, for each critic's Q-function, note that we assume that each critic knows the strategies of other actors. Considering the privacy issues, the UEs may not be willing to share their offloading strategies with each other. Therefore, to meet this requirement, we maintain the estimates of other agents' strategies. Concretely, we use $\hat{\mu}_{\phi_j^i}$ to denote the estimation of $Agent_i$'s strategy to $Agent_j$, where ϕ_j^i denotes the parameter of the estimation strategy. We take advantage of maximum likelihood estimation to estimate the strategy, and we add an entropy of the policy distribution H to it so that the policy won't be too certain. Then the loss function can be calculated by:

$$L(\phi_j^i) = -E_{o_j, a_j} [\log \hat{\mu}_{\phi_j^i}(a_j | o_j) + \lambda H(\hat{\mu}_{\phi_j^i})]. \quad (20)$$

Through minimizing this cost function, we can obtain an estimate of the strategy of other agents. Then, the Eq. (19) can be reformulated as:

$$\hat{y} = r_i + \gamma Q_i^{\mu'}(x', \mu_i^1(o_1), \mu_i^2(o_2), \dots, \mu_i^n(o_n)). \quad (21)$$

As mentioned above, since each agent's strategy is constantly updated in the multi-agent setting, the underlying environment is not stable for a particular agent anymore. In the face of the competitive tasks proposed in this article that may compete for bandwidth and computing resources, the agent may learn an overly strong and fragile strategy. The overly strong strategy is only effective for the current strategy adopted by other agents, so when other agents update their

TABLE 2
Simulation Parameters Configuration

Simulation Parameter	Value
Bandwidth of the wireless channel between UEs and LEOs	15-30 MHz
Bandwidth of the wireless channel between LEOs and cloud servers	50-100 MHz
Computation tasks' data size	7-15Mb
Amount of CPU cycles demanded by task	$0.5 - 0.9 \times 10^9$
Transmission power of UEs	50-100 mW
Transmission power of LEOs	200-300mW
Computational ability of UEs	0.1-1GHz
Computational ability of LEOs	5-6 GHz
Computational ability of cloud server	14-18GHz
Energy cost of UEs	0.5-1 J/GHz
Energy cost of LEOs	0.3-0.5 J/GHz
Energy cost of cloud server	0.1-0.3 J/GHz

strategies, the effectiveness of this strong strategy may become very poor. To solve this problem, we train K in different strategies for each agent. That is, the strategy μ_i of the i -th agent is a set of K sub-strategies. During every training episode, we just use one sub-strategy $\mu_i^{(k)}$. For each agent, we aim to maximize the total reward of its strategy set.

$$\nabla_{\theta_i^{(k)}} J(\mu_i) = \frac{1}{K} E_{x,a \sim D} [\nabla_{\theta_i^{(k)}} \mu_i^{(k)}(a_i | o_i) \text{Grad}_Q],$$

where $\text{Grad}_Q = \nabla_{a_i} Q_i^\mu(x, a_1, \dots, a_n) \big|_{a_i = \mu_i^{(k)}(o_i)}$ (22)

The multi-agent reinforcement learning aided computation offloading algorithm is shown in Algorithm 1.

Algorithm 1. The MADRL Algorithm for Computation Offloading in SMEC

- 1: Randomly initialize the parameters of the critic network and the actor network
- 2: Initialize the parameters of the target network and replay buffer R
- 3: **for** $episode = 1$ to $episodes - amount$ **do**
- 4: Initialize the SMEC scenario with randomly parameters
- 5: **for** $step = 1$ to $max - episode - length$ **do**
- 6: Each IoT device get its local observation o
- 7: Each IoT device chooses an action a according to o and current policy
- 8: Each IoT device receives the immediate reward r from environment, and obtains the next observation o'
- 9: Save (o, a, r, o') to the replay buffer R
- 10: **end for**
- 11: **for** agent $k = 1$ to $agent - number$ **do**
- 12: Randomly sample a minibatch (o_j, a_j, r_j, x'_j) from the replay buffer R
- 13: Update critic network through minimizing the loss function $(\theta) = E_{x,a,r,x'} [(Q_i^\mu(x, a_1, a_2, \dots, a_n) - y)^2]$
- 14: Update actor network by the function $\nabla_{\theta_i} J(\mu_i) = E_{x,a \sim D} [\nabla_{\theta_i} \mu_i(a_i | o_i) \text{Grad}_Q]$, where $\text{Grad}_Q = \nabla_{a_i} Q_i^\mu(x, a_1, \dots, a_n) \big|_{a_i = \mu_i(o_i)}$
- 15: **end for**
- 16: Update the parameters of the target network of the critic and the agents $\theta_i^{Q'} \leftarrow \tau \theta_i^Q + (1 - \tau) \theta_i^{Q'}$
 $\theta_i^{\mu'} \leftarrow \tau \theta_i^\mu + (1 - \tau) \theta_i^{\mu'}$
- 17: **end for**

TABLE 3
MADRL Parameters Configuration

Parameter	MADRL
Maximum step per episode	25
Maximum episode number	15000
Discount factor (γ)	0.96
number of network layers	3
Learning rate for Adam optimizer (α)	0.01
Experience batch size	500 episodes
number of neurons per layer	64
Target network	True
Action Space	Discrete
Save model rate	1000
batch size	1024

5 SIMULATION RESULTS

In this section, we present simulation results to evaluate the feasibility and performance of our algorithm. We run the simulation on a host with a GPU installed, where the CPU is an Intel(R) Xeon(R) Gold 5218R CPU @ 2.10 GHz. The size of the hard disk is 512 G. We set up the simulation environment with python 3.5.4 and we implement the algorithm by TensorFlow 1.8.0. In our experiment, the bandwidth of the wireless channel between UEs and LEOs is equally distributed between 15 MHz to 30 MHz. The bandwidth of wireless channels between LEOs and cloud servers is equally distributed between 50 MHz to 100 MHz. The computation tasks' data size is equally distributed between 7Mb to 15Mb. The amount of CPU cycles demanded by the task is equally distributed between 0.5×10^9 to 0.9×10^9 .

5.1 Experiment Configuration

In our experiment, with the cloud server on the earth, the LEOs are evenly distributed within the range of 500 km to 2000 km from the cloud server, and the UEs are evenly distributed within the range of 500 km to 2000 km from the LEOs. To evaluate the convergence performance of our algorithm under different load conditions, we set up the following four experiment topology. The first topology contains 1 cloud server, 1 LEO, and 2UEs. The second topology contains 1 cloud server, 2 LEOs, and 4UEs. The third topology contains 1 cloud server, 3 LEOs, and 9UEs. The fourth topology contains 1 cloud server, 4 LEOs, and 16UEs. The detailed parameters in the experiment are given in Table 2 and We randomly choose the bandwidth, noise power, and channel gain within the range.

5.2 Convergence Analysis

Besides, the parameters of our MADRL algorithm is given in Table 3 and the architecture of MADRL algorithm is shown in Fig. 5.

5.3 Baseline Algorithms

In this paper, we set three baseline algorithms to evaluate the effectiveness of our algorithm. The first baseline algorithm is Deep Deterministic Policy Gradient (DDPG) [34]. DDPG is a classical single-agent actor-critic reinforcement learning algorithm, where the actor and critic share the same state space. The connection between actor and critic in DDPG is as follows: The actor observes the environment

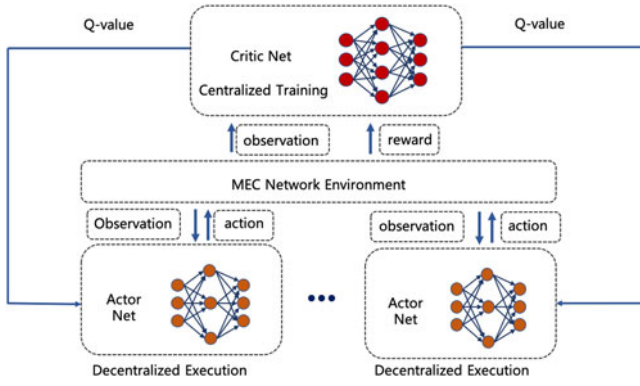


Fig. 5. MADRL algorithm architecture.

and chooses an exact action while the critic takes the actor's action and observation as input and outputs the value of the chosen action. The goal of the critic update is to accurately evaluate the value of the actor's actions, while the goal of the actor update is to maximize the critic's evaluation. The second baseline algorithm is the Local Execution algorithm (LE) [35]. In LE, every user chooses to execute the tasks locally. The third baseline algorithm is Random Action (RA) [36]. In RA, the user randomly selects the offloading decision to execute tasks and randomly allocates transmission power.

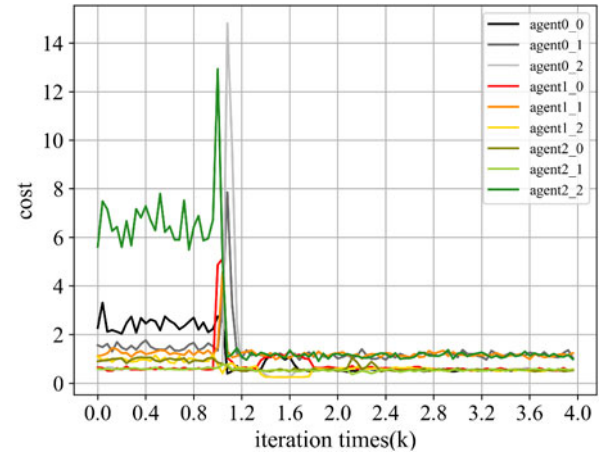
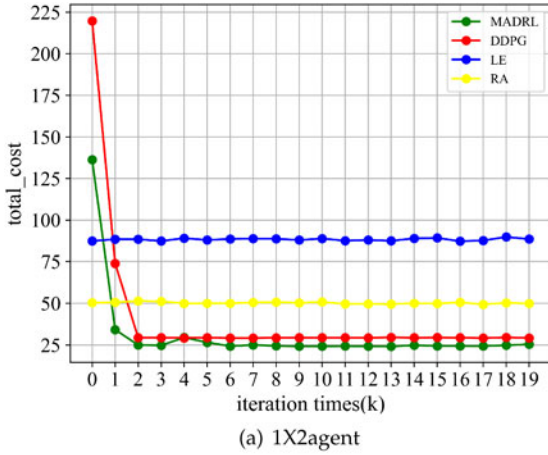
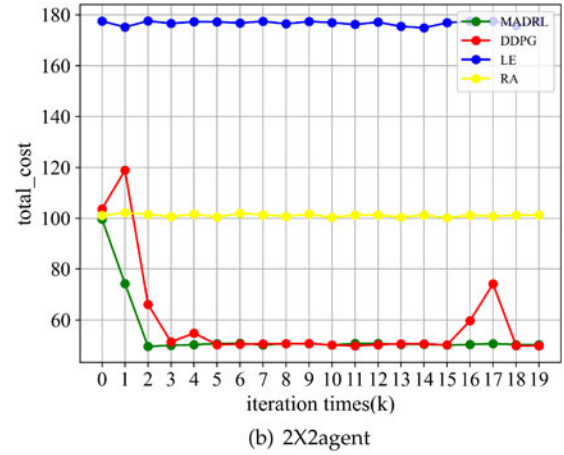


Fig. 6. Cost per step of each UE under 3rd Topology.

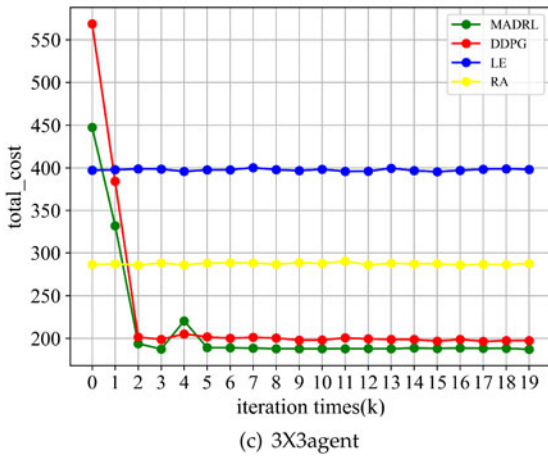
The convergence of our algorithm is firstly evaluated and presented in this section. We select the third network topology, where the network consists of a cloud server, 3 LEOs, and 9 UEs. As shown in Fig. 6, we notice that the learning cost can quickly converge to stable at approximately 2 k iteration times and the cost function is defined in Eq. (15). And it is worth mentioning that each UE has one computation task to process in every iteration. For each task, the actor will choose the offloading location and determine the transmission power.



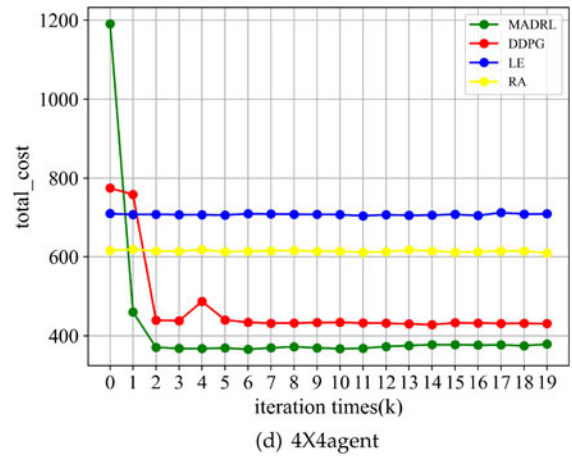
(a) 1X2agent



(b) 2X2agent



(c) 3X3agent



(d) 4X4agent

Fig. 7. Total reward under different topologies.

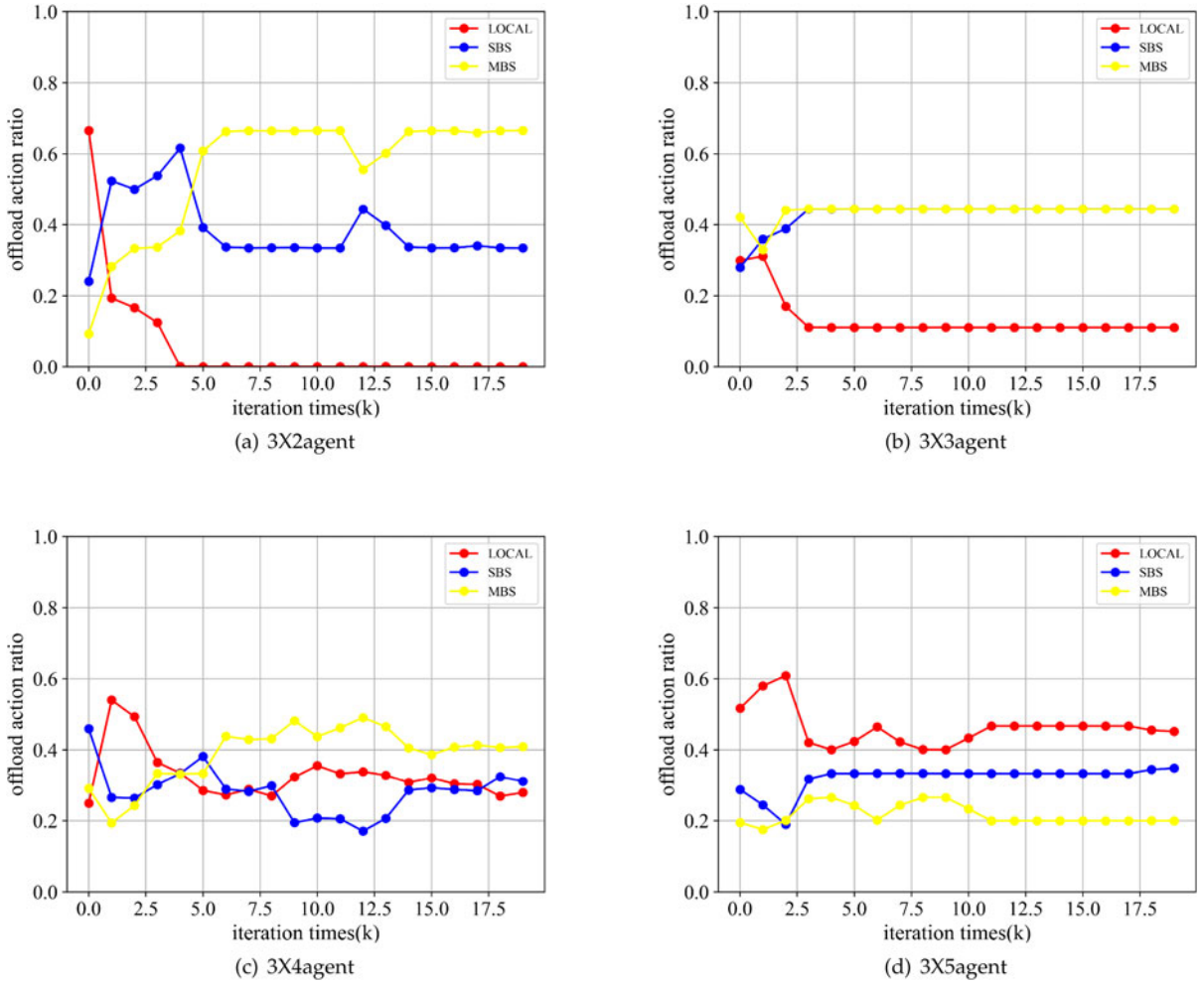


Fig. 8. Offloading behavior under different topologies.

for the UE. In addition, we can conclude that the convergence value of different UE is not much different. The reason is that the centralized critic, which is augmented with the global network state, can effectively alleviate the distributed agent learning procedure. These experimental results illustrate that our proposed algorithm presents a good convergence and fairness.

5.4 Performance Analysis

In order to fully demonstrate the effect of the proposed algorithm, we compare its performance with the baseline algorithms in different topologies by comparing the total cost, which is the sum of all the UEs' costs in one thousand iteration times. We set four topologies where the number of LEOs and UEs differ from each other and we denote them with the figure note such as 2X3agent, which means the topology contains 2 LEOs and each LEO is connected with 3 UEs. As shown in Fig. 7, when the number of UEs is small, we notice the reinforcement learning-based algorithms (i.e., MADRL, DDPG) exhibit better performance compared to the two other algorithms. The reason is that reinforcement learning can automatically adapt to environmental dynamics and learn optimal strategy directly from experience. As the number of UEs increases, the MADRL algorithm presents performance benefits gradually reflected compared to the DDPG algorithm. This performance gain is mainly because MADRL

adopts a centralized "critic" that maintains the global state and joint action of all UEs to revise each UE's training process. This revised signal can effectively enhance the distributed agent collaboration ability, and therefore increase the whole network utility. In contrast, the DDPG adopts a selfish scheme to update the offloading strategy only according to local reward rather than global reward. While this selfish scheme will bring high returns quickly, it leads to the non-convergence of the learning procedure, especially with the number of agents increasing.

5.5 Behavior Analysis

Then, we demonstrate the behavior of each UE during the training process and show how the proposed algorithm guide the offloading behavior in different load conditions. In this experiment, we set up a network scenario, which contains one cloud server and three LEOs. Then, we increase the number of UEs under each LEO to analyze each UE's offloading behavior. As shown in Fig. 8, we notice that as the number of UEs increasing, the UEs are more willing to execute the tasks locally rather than in the cloud server. This is caused by that too many UEs will carry the computing pressure to cloud servers and LEO.

Besides, we notice that the willingness to offload tasks to LEO slightly increased at first and then decreased. That is because that when UE's number is increasing, the

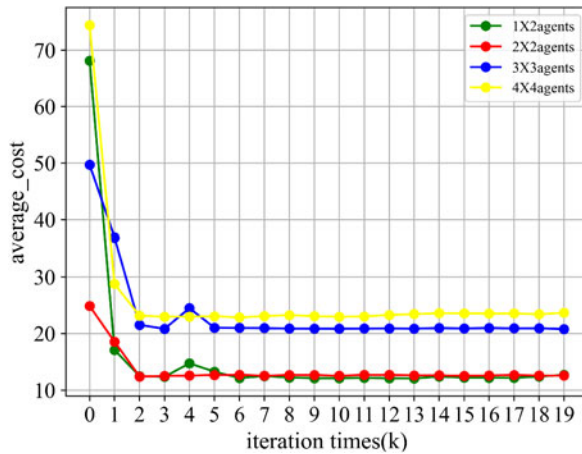


Fig. 9. Average cost under different topologies.

computation pressure of the MEC server on the cloud server increases as well. Then, the cost of offloading tasks to the cloud servers grows above the cost of offloading to LEO. However, with the UE's number continuously increasing, the MEC server's pressure on LEO also increases, and therefore significantly increases the offloading cost. Then, UEs are more likely to execute the tasks locally. Therefore, we can tell that the algorithm can help the UEs avoid congestion and choose the optimal offloading decision for it.

5.6 Average Cost Vs. UE Numbers

Next, we evaluate the performance of our algorithm under different UE numbers. As shown in Fig. 9, the algorithm has achieved good performance in different scenarios. As the number of UEs increasing, the MEC server's load in the network increases and therefore increases the processing delay. Thus, the average cost increases slightly as the amount of UEs increasing.

6 CONCLUSION

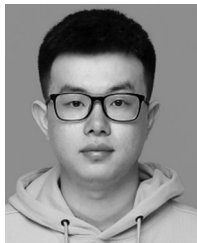
In this paper, we investigate the characteristics of computation offloading under the LEO network architecture and model the UE's offloading decision problem as a distributed decision problem, which, we think, is closer to the real situation of the network. We then proposed a joint three-tier computation offloading framework, including offloading location selection and transmission energy allocation issues. We chose to apply the multi-agent deep reinforcement learning (MADRL) algorithm in our proposed architecture. In the algorithm, an actor is deployed on each UE. It can make decisions independently based on local observations. And a logically centralized critic platform is deployed on the cloud servers. The platform collects the information of the network, MEC servers, and UEs and gives updating advice to agents, aiding them to maximize the cumulative expected reward. The experiment results illustrated that the algorithm can converge quickly and outperforms other algorithms.

REFERENCES

- [1] M. Centenaro, C. E. Costa, F. Granelli, C. Sacchi, and L. Vangelista, "A survey on technologies, standards and open challenges in satellite IoT," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1693–1720, Jul.–Sep. 2021.

- [2] Q. Li et al., "Service coverage for satellite edge computing," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 695–705, Jan. 2022.
- [3] S. Yu, X. Gong, Q. Shi, X. Wang, and X. Chen, "EC-SAGINs: Edge computing-enhanced space-air-ground integrated networks for internet of vehicles," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5742–5754, Apr. 2022.
- [4] Q. Tang, Z. Fei, B. Li, and Z. Han, "Computation offloading in LEO satellite networks with hybrid cloud and edge computing," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 9164–9176, Jun. 2021.
- [5] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016.
- [6] R. Xie, Q. Tang, Q. Wang, X. Liu, F. R. Yu, and T. Huang, "Satellite-terrestrial integrated edge computing networks: Architecture, challenges, and open issues," *IEEE Netw.*, vol. 34, no. 3, pp. 224–231, May/Jun. 2020.
- [7] Y. Gong, H. Yao, J. Wang, M. Li, and S. Guo, "Edge intelligence-driven joint offloading and resource allocation for future 6G industrial Internet of Things," *IEEE Trans. Netw. Sci. Eng.*, to be published, doi: [10.1109/TNSE.2022.3141728](https://doi.org/10.1109/TNSE.2022.3141728).
- [8] T. Mai, H. Yao, J. Xu, N. Zhang, Q. Liu, and S. Guo, "Automatic double-auction mechanism for federated learning service market in Internet of Things," *IEEE Trans. Netw. Sci. Eng.*, to be published, doi: [10.1109/TNSE.2022.3170336](https://doi.org/10.1109/TNSE.2022.3170336).
- [9] C. Xian, Y.-H. Lu, and Z. Li, "Adaptive computation offloading for energy conservation on battery-powered systems," in *Proc. Int. Conf. Parallel Distrib. Syst.*, 2007, pp. 1–8.
- [10] J. Liu, K. Kumar, and Y.-H. Lu, "Tradeoff between energy savings and privacy protection in computation offloading," in *Proc. ACM/IEEE Int. Symp. Low-Power Electron. Des.*, 2010, pp. 213–218.
- [11] S. Chen, X. Zhu, H. Zhang, C. Zhao, G. Yang, and K. Wang, "Efficient privacy preserving data collection and computation offloading for fog-assisted IoT," *IEEE Trans. Sustain. Comput.*, vol. 5, no. 4, pp. 526–540, Oct.–Dec. 2020.
- [12] X. Xu, C. He, Z. Xu, L. Qi, S. Wan, and M. Z. A. Bhuiyan, "Joint optimization of offloading utility and privacy for edge computing enabled IoT," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 2622–2629, Apr. 2020.
- [13] T. Mai, H. Yao, N. Zhang, L. Xu, M. Guizani, and S. Guo, "Cloud mining pool aided blockchain-enabled Internet of Things: An evolutionary game approach," *IEEE Trans. Cloud Comput.*, to be published, doi: [10.1109/TCC.2021.3110965](https://doi.org/10.1109/TCC.2021.3110965).
- [14] J. Wang, J. Hu, G. Min, W. Zhan, Q. Ni, and N. Georgalas, "Computation offloading in multi-access edge computing using a deep sequential model based on reinforcement learning," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 64–69, May 2019.
- [15] A. Alsharoa and M.-S. Alouini, "Improvement of the global connectivity using integrated satellite-airborne-terrestrial networks with resource optimization," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5088–5100, Aug. 2020.
- [16] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
- [17] J. Liu, X. Du, J. Cui, M. Pan, and D. Wei, "Task-oriented intelligent networking architecture for the space-air-ground-aqua integrated network," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5345–5358, Jun. 2020.
- [18] J. Han, H. Wang, S. Wu, J. Wei, and L. Yan, "Task scheduling of high dynamic edge cluster in satellite edge computing," in *Proc. IEEE World Congr. Serv.*, 2020, pp. 287–293.
- [19] Z. Zhang, W. Zhang, and F.-H. Tseng, "Satellite mobile edge computing: Improving QoS of high-speed satellite-terrestrial networks using edge computing techniques," *IEEE Netw.*, vol. 33, no. 1, pp. 70–76, Jan./Feb. 2019.
- [20] Y. Wang, J. Yang, X. Guo, and Z. Qu, "A game-theoretic approach to computation offloading in satellite edge computing," *IEEE Access*, vol. 8, pp. 12510–12520, 2019.
- [21] F. Guo, H. Zhang, H. Ji, X. Li, and V. C. Leung, "An efficient computation offloading management scheme in the densely deployed small cell networks with mobile edge computing," *IEEE/ACM Trans. Netw.*, vol. 26, no. 6, pp. 2651–2664, Jun. 2018.
- [22] J. Guo, H. Zhang, L. Yang, H. Ji, and X. Li, "Decentralized computation offloading in mobile edge computing empowered small-cell networks," in *Proc. IEEE Globecom Workshops*, 2017, pp. 1–6.
- [23] L. Huang, S. Bi, and Y.-J. A. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 11, pp. 2581–2593, Nov. 2020.

- [24] Z. Song, Y. Hao, Y. Liu, and X. Sun, "Energy-efficient multiaccess edge computing for terrestrial-satellite Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 14202–14218, Sep. 2021.
- [25] W. Chen *et al.*, "Cooperative and distributed computation offloading for blockchain-empowered industrial Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8433–8446, May 2019.
- [26] Z. Zhao *et al.*, "A novel framework of three-hierarchical offloading optimization for MEC in industrial IoT networks," *IEEE Trans. Ind. Informat.*, vol. 16, no. 8, pp. 5424–5434, Aug. 2019.
- [27] W. Zhan *et al.*, "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.
- [28] C. Liu, F. Tang, Y. Hu, K. Li, Z. Tang, and K. Li, "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1603–1614, Jul. 2020.
- [29] H. Zhang, J. Guo, L. Yang, X. Li, and H. Ji, "Computation offloading considering fronthaul and backhaul in small-cell networks integrated with mec," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2017, pp. 115–120.
- [30] G. Liu, F. R. Yu, H. Ji, and V. C. Leung, "Virtual resource management in green cellular networks with shared full-duplex relaying and wireless virtualization: A game-based approach," *IEEE Trans. Veh. Technol.*, vol. 65, no. 9, pp. 7529–7542, Sep. 2015.
- [31] J. Dai and S. Wang, "Clustering-based spectrum sharing strategy for cognitive radio networks," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 1, pp. 228–237, Jan. 2017.
- [32] G. Shani, J. Pineau, and R. Kaplow, "A survey of point-based pomdp solvers," *Auton. Agents Multi-Agent Syst.*, vol. 27, no. 1, pp. 1–51, 2013.
- [33] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Oct.–Dec. 2017.
- [34] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [35] M. Liu and Y. Liu, "Price-based distributed offloading for mobile-edge computing with computation capacity constraints," *IEEE Wireless Commun. Lett.*, vol. 7, no. 3, pp. 420–423, Jun. 2018.
- [36] Y. Liu, F. R. Yu, X. Li, H. Ji, and V. C. Leung, "Distributed resource allocation and computation offloading in fog and cloud networks with non-orthogonal multiple access," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12137–12151, Dec. 2018.



Zeyu Qin (Student Member, IEEE) is currently working toward the master's degree in the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing. His research interests include future network architecture, programmable data plane, network artificial intelligence, multi-agent system, network resource allocation and dedicated networks.



Haipeng Yao (Senior Member, IEEE) is received the PhD in the Department of Telecommunication Engineering, University of Beijing University of Posts and Telecommunications, in 2011. He is a professor in Beijing University of Posts and Telecommunications. His research interests include future network architecture, network artificial intelligence, networking, space-terrestrial integrated network, network resource allocation and dedicated networks. He has published more than 150 papers in prestigious peer-reviewed journals and conferences. He has served as an editor of *IEEE Network*, *IEEE Transactions on Sustainable Computing*, *IEEE Access*, and a guest editor of *IEEE Open Journal of the Computer Society* and *Springer Journal of Network and Systems Management*. He has also served as a member of the technical program committee as well as the Symposium chair for a number of international conferences, including IWCMC 2019 Symposium Chair, ACM TUR-C SIGSAC2020 Publication Chair.



Tianle Mai (Member, IEEE) is currently working toward the PhD degree in the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing. His research interests include future network architecture, network artificial intelligence, multi-agent system, space-terrestrial integrated network, network resource allocation and dedicated networks.



Di Wu (Student Member, IEEE) (Student Member, IEEE) received the master's degree in computer system engineering from the University of Houston, in 2020. He is currently working toward the PhD degree in Beijing University of Posts and Telecommunications. His research interests include future network architecture, network artificial intelligence, space-terrestrial integrated network, and Big Data.



Ni Zhang received the PhD with the Institute of Computing Technology, Chinese Academy of Sciences, in 2007. He currently serves with the Sixth Research Institute of China Electronic Corporation, Beijing, China. His research interests are in the area of future Internet architecture, network security and artificial intelligence.



Song Guo (Fellow, IEEE) is a full professor and associate head with the Department of Computing, The Hong Kong Polytechnic University. His research interests are mainly in the areas of Big Data, cloud computing, mobile computing, and distributed systems. He is the recipient of the 2019 IEEE TCBDBest Conference paper award, 2018 IEEE TCGCC Best Magazine paper award, 2017 IEEE Systems Journal Annual best paper award, and other 6 best paper awards from IEEE/ACM conferences. His work was also recognized by the

2016 annual best of computing: Notable Books and Articles in Computing in ACM Computing Reviews. He was an IEEE ComSoc Distinguished Lecturer (2017–2018) and served in IEEE ComSoc Board of Governors (2018–2019). He has also served as General and Program chair for numerous IEEE conferences. He is an IEEE editor-in-chief of IEEE Open Journal of the Computer Society, and associate editor of *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Sustainable Computing*, and *IEEE Transactions on Green Communications and Networking*.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**